

VLA Drone Agent Experiment Report

1 任务概述

本项目实现了一个基于自然语言指令的最小闭环无人机仿真 Agent。系统能够将中文或英文任务指令解析为结构化动作序列，并在 PyBullet 仿真环境中完成目标搜索、移动、悬停、降落、状态检查、失败处理和实验记录。

示例任务：

1 | 起飞，找到红色目标，飞到它上方1米处悬停5秒，然后降落

系统将其解析为如下动作序列：

```
1 | [  
2 |   {"action": "takeoff", "altitude": 1.5},  
3 |   {"action": "search", "target": "red"},  
4 |   {"action": "move_above", "target": "red", "height": 1.0},  
5 |   {"action": "hover", "duration": 5},  
6 |   {"action": "land"}  
7 | ]
```

本项目重点不是实现复杂的无人机动力学或大规模端到端模型，而是在有限时间内完成一个可运行、可解释、可复现、可迭代的最小闭环 demo，并通过日志、视频、轨迹和实验结果验证系统行为。

2 系统架构

系统整体流程如下：

自然语言任务 →

1. Planner：将自然语言任务解析为结构化 action list。
2. Schema Validation：检查 action 类型、必要参数、目标颜色和参数范围。
3. Safety Check：检查任务是否超出仿真工作空间边界。
4. Perception：定位颜色目标并输出目标位置。
5. Controller：控制无人机执行起飞、移动、悬停和降落。
6. Agent Loop：按 action list 执行任务、检查状态、处理失败和 fallback。
7. Recorder：保存 demo.mp4、trajectory.csv 和 events.jsonl。
8. Evaluation：运行 10 条自然语言任务并生成 results.csv。

主要代码结构：

```
1 | vla-drone-agent/  
2 | └─ agent/  
3 |   └─ planner.py           # 规则 Planner: 中英文指令解析  
4 |   └─ llm_planner.py       # 可选 LLM Planner  
5 |   └─ planner_backend.py    # Planner 后端选择:auto/rule/llm 与 fallback  
6 |   └─ schema.py            # Pydantic action schema 校验  
7 |   └─ safety.py             # 工作空间边界与安全检查  
8 |   └─ replanner.py          # 失败后的安全 fallback plan
```

```

9 |
10 | └─ perception/
11 |   └─ color_detector.py      # 颜色目标感知接口
12 |   └─ camera.py             # 相机/观测扩展预留
13 |
14 | └─ sim/
15 |   └─ world.py               # PyBullet 世界与彩色目标
16 |   └─ drone.py               # 简化无人机模型
17 |   └─ controller.py         # 起飞、移动、悬停、降落控制
18 |   └─ recorder.py           # 视频、轨迹与事件日志记录
19 |
20 | └─ experiments/
21 |   └─ tasks.json             # 10 条自然语言评估任务
22 |   └─ evaluate.py           # 批量评估脚本
23 |
24 | └─ outputs/
25 |   └─ demo.mp4              # 单任务演示视频
26 |   └─ trajectory.csv        # 无人机轨迹与控制误差
27 |   └─ events.jsonl          # Agent Loop 事件日志
28 |   └─ results.csv           # 批量实验结果
29 |
30 | └─ docs/
31 |   └─ dev_log.md             # 每日开发记录
32 |   └─ ai_usage.md            # AI 使用说明
33 |   └─ report.pdf             # 实验报告
34 |   └─ research_note.pdf      # Research Note
35 |
36 | └─ run_demo.py              # 单条自然语言任务 demo 入口与 Agent Loop
37 | └─ requirements.txt          # Python 依赖
38 | └─ README.md

```

3 Planner 设计

本项目实现了两类 Planner：

1. Rule Planner;
2. 可选 LLM Planner。

3.1 Rule Planner

Rule Planner 位于 `agent/planner.py`，用于稳定解析受控自然语言表达，例如：

- 起飞 / take off;
- 找到红色目标 / find the red target;
- 飞到目标上方 / move above target;
- 悬停 5 秒 / hover 5 seconds;
- 降落 / land;
- move to x=1 y=-1 z=1.2。

3.2 LLM Planner

LLM Planner 位于 `agent/llm_planner.py`，支持 OpenAI-compatible API。

3.3 Planner 模式

系统提供三种 Planner 模式：

- rule：只使用规则 Planner
- llm：只使用 LLM Planner
- auto：优先使用 LLM Planner，失败时自动回退到 Rule Planner

默认采用 `auto` 模式。这样既可以利用 LLM 的自然语言理解能力，又避免系统完全依赖外部 API。当 LLM 不可用、输出非法 JSON、生成非法 action 或 safety check 不通过时，系统会自动 fallback 到 Rule Planner。

三种模式的运行方式如下：

① 默认 `auto` 模式：

```
1 python run_demo.py --task "起飞，找到红色目标，飞到它上方1米处悬停5秒，然后降落"
```

② 强制使用规则 Planner：

```
1 python run_demo.py --planner rule --task "起飞，找到红色目标，飞到它上方1米处悬停5秒，然后降落"
```

③ 强制使用 LLM Planner：

```
1 python run_demo.py --planner llm --task "起飞，找到红色目标，飞到它上方1米处悬停5秒，然后降落"
```

其中，`llm` 模式要求提前配置 LLM API Key；如果希望在 LLM 不可用时自动回退到规则 Planner，应使用默认的 `auto` 模式。

4 Schema Validation 与 Safety 设计

Planner 输出的 action list 不会直接进入控制器，而是必须经过结构化校验和安全检查。

`agent/schema.py` 使用 Pydantic 定义 action schema，限制合法 action 包括：

```
1 takeoff
2 search
3 move_to
4 move_above
5 hover
6 land
```

同时检查必要参数，例如：

- `takeoff` 必须包含 `altitude`；
- `search` 必须包含 `target`；

- `move_to` 必须包含三维 `position`;
- `move_above` 必须包含 `target` 和 `height`;
- `hover` 必须包含 `duration`。

`agent/safety.py` 实现工作空间边界检查。当前仿真空间设置为：

```
1 x: [-3.0, 3.0]
2 y: [-3.0, 3.0]
3 z: [0.0, 3.0]
```

例如：

```
1 take off, move to x=5 y=0 z=1, hover 2 seconds, then land
```

会在执行前被拒绝，失败原因记录为：

```
1 planning_failed:safety_violation
```

该设计避免 Planner 或 LLM 输出错误动作导致控制器直接崩溃。

5 Perception 设计

Perception 模块位于 `perception/color_detector.py`，支持 red、blue、green 三类目标定位。

当前 demo 使用 PyBullet simulator observation 获取目标世界坐标。也就是说，仿真世界中保存了每个颜色目标的位置，Perception 模块通过统一接口返回：

```
1 color
2 position
3 source = sim_observation
```

例如，红色目标检测结果为：

```
1 color: red
2 position: (1.5, 0.8, 0.05)
3 source: sim_observation
```

对于 `move_above(red, height=1.0)`，系统将目标位置转换为控制目标：

```
1 (red_x, red_y, 1.0)
```

需要说明的是，当前 Perception 并未实现完整 RGB 图像检测和图像坐标到世界坐标的反投影。这是为了保证最小闭环稳定可复现。后续可以将该接口替换为基于 PyBullet camera image、OpenCV 颜色分割和深度信息的图像级感知模块。

6 Controller 与 Agent Loop

仿真环境由 `sim/world.py` 创建，包含地面、红/蓝/绿目标物和简化无人机模型。无人机模型由 `sim/drone.py` 定义，使用可见的 box 和机臂结构表示。

控制器位于 `sim/controller.py`，采用简化位置控制，支持：

```
1 | takeoff
2 | move_to
3 | hover
4 | land
```

控制器每个 step 将无人机朝目标位置移动固定距离。到达判定阈值为：

```
1 | arrival_threshold = 0.08 m
```

当无人机与目标位置距离小于该阈值时，认为动作完成。

主 Agent Loop 位于 `run_demo.py`，负责：

1. 接收自然语言任务；
2. 调用 Planner 得到 action list；
3. 调用 Perception 获取目标位置；
4. 调用 Controller 执行动作；
5. 在每个动作后检查状态；
6. 记录事件日志和轨迹；
7. 在执行失败时触发 fallback。

每个动作执行后都会记录：

```
1 | success
2 | final_error
3 | steps
4 | reason
```

这些信息会写入 `outputs/events.jsonl`。

7 失败处理与 Fallback

本项目区分两类失败：

7.1 Planning / Schema / Safety 阶段失败

这类失败发生在无人机执行前，例如：

- 无法解析指令；
- 不支持的颜色；
- action 缺失参数；
- 越界任务。

系统不会让无人机起飞，而是直接拒绝任务并记录：

```
1 | planning_failed:unparseable_instruction
2 | planning_failed:unsupported_target_color
3 | planning_failed:safety_violation
4 | planning_failed:schema_validation
```

7.2 执行阶段失败

这类失败发生在任务已经合法、但执行过程中无法完成，例如：

- 目标颜色合法，但环境中目标缺失；
- Controller 执行动作超过最大步数。

此时系统会触发安全 fallback replanning。当前 fallback plan 为：

```
1 [
2   {"action": "hover", "duration": 1.0},
3   {"action": "land"}
4 ]
```

也就是说，系统不会继续执行原始任务，而是短暂悬停后安全降落。事件日志中会记录：

```
1 fallback_started
2 fallback_action_started
3 fallback_action_finished
4 fallback_finished
```

本项目没有实现复杂语义级重新规划，例如重新搜索其他目标或生成新任务计划；当前实现属于安全恢复型 fallback replanning，满足最小闭环任务对 replan 或 fallback 的要求。

8 Recorder 与输出文件

系统运行后会生成以下输出：

- `outputs/demo.mp4`：PyBullet 仿真视频；
- `outputs/trajectory.csv`：无人机轨迹、目标位置和控制误差；
- `outputs/events.jsonl`：任务事件日志；
- `outputs/results.csv`：批量实验结果。

`trajectory.csv` 包括：

```
1 time
2 x
3 y
4 z
5 target_x
6 target_y
7 target_z
8 error
```

`events.jsonl` 记录任务开始、动作开始、目标检测、动作完成、失败原因、fallback 和任务结束等事件。

9 实验设计

本项目设计了 10 条自然语言任务，覆盖以下情况：

1. 正常红色目标任务；

2. 正常蓝色目标任务;
3. 正常绿色目标任务;
4. 英文任务;
5. move_to 坐标任务;
6. 红色目标缺失;
7. 蓝色目标缺失;
8. 不支持的颜色目标;
9. 越界 move_to 任务;
10. 无法解析的自然语言任务。

实验通过以下命令运行:

```
1 python experiments/evaluate.py --task-file experiments/tasks.json
```

结果保存到:

```
1 outputs/results.csv
```

10 实验结果

本次实验共运行 10 条任务, 结果如下:

```
1 Total tasks: 10
2 Success rate: 50.00%
3 Average duration: 5.479s
4 Average final position error on success: 0.0678m
```

失败原因分类如下:

```
1 target_not_found:red: 1
2 target_not_found:blue: 1
3 planning_failed:unsupported_target_color: 1
4 planning_failed:safety_violation: 1
5 planning_failed:unparseable_instruction: 1
```

成功任务包括 red、blue、green 三类目标任务、英文任务和 move_to 坐标任务。失败任务主要用于验证系统对异常情况的处理能力。

11 失败案例分析

11.1 目标缺失: target_not_found:red

任务要求寻找红色目标, 但实验中通过 `missing_targets=["red"]` 移除了红色目标。Planner 和 Schema 均认为任务合法, 但 Perception 在执行阶段无法找到 red 目标。

系统行为:

```
1 | search red failed
2 | record target_not_found:red
3 | trigger fallback
4 | hover briefly
5 | land
```

该案例验证了 Agent Loop 在执行阶段感知失败时能够记录原因并执行安全 fallback。

11.2 越界任务：planning_failed:safety_violation

任务为：

```
1 | take off, move to x=5 y=0 z=1, hover 2 seconds, then land
```

Planner 成功解析出 `move_to` 动作，但 Safety 检查发现 $x=5$ 超出工作空间边界。因此系统在执行前拒绝任务。

该案例验证了 Safety 模块能够防止危险任务进入控制器。

11.3 不支持颜色：planning_failed:unsupported_target_color

任务为：

```
1 | 起飞，找到青色目标，然后降落
```

当前系统只支持 red、blue、green 三类颜色目标。因此 Planner 检测到 unsupported target color，并在执行前拒绝任务。

该案例说明系统不会静默忽略不支持目标，而是显式记录失败原因，便于后续实验分析。

12 当前简化与不足

当前 demo 做了以下简化：

- Perception 使用 simulator observation，而非真实图像检测。
- 无人机控制采用简化位置控制，而非真实四旋翼动力学或电机控制。
- 仿真环境较简单，没有动态障碍物、风扰动或传感器噪声。
- Rule Planner 只能覆盖受控自然语言表达。
- LLM Planner 是可选模块，仍可能输出不稳定结果，因此必须经过 schema 和 safety 检查。
- Fallback 采用保守的 hover + land，没有实现复杂语义级 replan。

13 真实无人机迁移需要补充的能力

如果迁移到真实无人机，还需要补充：

- 真实 RGB / Depth 感知；
- 目标检测和跟踪；
- 图像坐标到世界坐标的转换；
- SLAM 或外部定位系统；
- 低层飞控和 PID / MPC 控制；

- 避障与路径规划;
- 安全 geofence;
- 通信延迟和硬件故障处理;
- 实机测试中的安全保护机制。

14 总结

本项目实现了一个语言驱动的无人机仿真 Agent 最小闭环。系统能够将自然语言任务解析为动作序列，在 PyBullet 中执行，并输出视频、轨迹、事件日志和实验结果。

项目重点体现了以下能力：

- 自然语言任务解析;
- LLM Planner 与规则 fallback;
- action schema 校验;
- safety check;
- 颜色目标感知;
- 简化无人机控制;
- 状态检查和失败处理;
- 批量实验评估;
- 可复现工程记录。